LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

Level 6 – Data and Web Development

Assessment Type

Semester 2025/26 Autumn

Student Name:Anurodh

Prasain

London Met

ID:23047512

College

ID:np01cp4a230012

Assignment Due Date: 24 dec 2025

Assignment Submission Date: 24 dec 2025

Submitted To: Shresha Rajbhandari

Word Count (Where Required):

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submissions and a mark of zero will be awarded.

1. Introduction

2. Initial Entity Relation Diagram

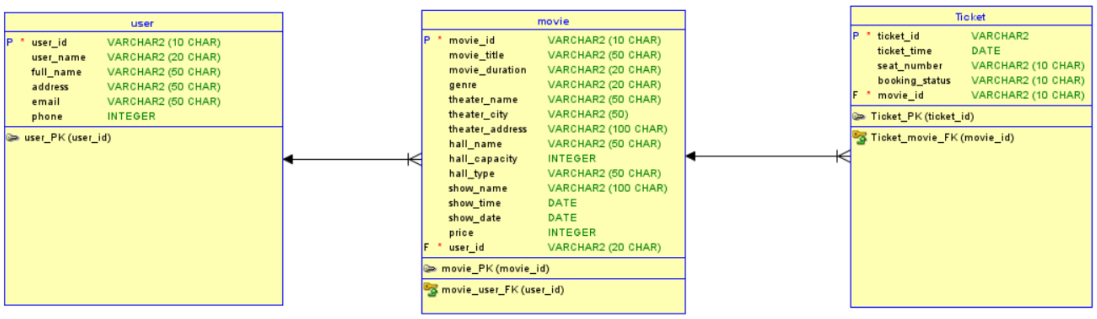


Figure 1: Initial ERD.

3. Normalization

3.1. Unnormalized Form (UNF)

According to the case study, the first unnormalized relation in terms of the Movie Ticketing System is represented as follows:

User

(**user_id**, user_name, user_email, user_phone, user_address,

{**movie_id**, movie_title, movie_duration, genre, release_date,

{**theater_id**, theater_name, theater_city, theater_address,

{**hall_id**, hall_name, capacity, hall_type,

{**show_id**, show_name, show_time, show_date, show_price,

{**ticket_id**, ticket_time, seat_no, status

}}}}}})

This structure represents a single flat view of the system in which there are several levels of data that exist in the User entity.

Identification of Repeating Groups

The UNF relation has several repeating and nested groups such as:

- A user can be linked to multiple movies.
- There are multiple theaters that can be identified with each movie.
- Multiple halls can be included in every theater.
- There are multiple shows that can be held in each hall.
- Each show can have multiple tickets.

These repeating groups are represented using curly brackets {} in the UNF definition.

Why UNF Violates First Normal Form (1NF)

A relation violates First Normal Form (1NF) if:

- Attributes are not atomic
- Repeating groups exist

The UNF relation violates 1NF because:

- Multiple values are stored within a single attribute group
- Nested structures exist inside the User entity
- Attributes such as movie details, theater details, show details, and ticket details are not atomic

Therefore, the UNF relation does not satisfy the requirements of 1NF.

Anomalies Present in UNF

Due to redundancy and lack of structure, the following anomalies exist

- Insertion anomaly: A new movie or theater cannot be inserted unless a user exists
- Update anomaly: Updating movie or hall details requires modifying multiple nested records
- Deletion anomaly: Deleting a user may result in loss of movie, show, or ticket information

3.2. First Normal Form (1NF)

To achieve 1NF:

- Repeating groups are removed
- All attributes are made atomic
- Separate relations are created for repeated data

Resulting 1NF Relations

USER - 1 (**user_id**, user_name, user_email, user_phone, user_address)

MOVIE - 1 (**user_id**, **movie_id**, movie_title, movie_duration, genre, release_date)

THEATER - 1 (**user_id, movie_id**, **theater_id**, theater_name, theater_city, theater_address)

HALL - 1 (**user_id, movie_id**, **theater_id**, **hall_id**, hall_name, capacity, hall_type)

SHOW - 1 (**user_id, movie_id**, **theater_id**, **hall_id**, **show_id**, show_name, show_time, show_date, show_price)

TICKET - 1 (**user_id, movie_id**, **theater_id**, **hall_id**, **show_id**, **ticket_id**, ticket_time, seat_no, status)

Each relation contains atomic values only, thus satisfying 1NF.

Anomalies and Difficulties

First Normal Form ascertains that every property is atomic and repeating collections are taken away. Nonetheless, although it meets 1NF, there are still several data anomalies and design challenges:

- Dependency is because of composite key primary.
- There is still redundancy of data.
- When the independent data cannot be inserted, we have insertion anomaly.
- Repetitive data is what causes update anomaly.
- Discard anomaly can lead to unwanted loss of data.

3.3. Second Normal Form (2NF)

A relation is said to be in Second normal form (2NF) provided:

- It already is in First Normal Form (1NF), and
- No non-key attribute is completely functional dependent on a partial of the composite primary key.

Determination of Functional Dependencies Partial and Full.

Once the UNF relations were converted to 1NF, each relation that resulted was looked at one attribute at a time trying to determine:

- Partial dependencies
- Full functional dependencies

This analysis will make sure that no non-key attribute is reliant on a subgroup of a composite primary key.

USER - 1

(<u>user_id</u>, user_name, user_email, user_phone, user_address)

Primary Key: user_id

Non-key attributes: user_name, user_email, user_phone, user_address

Functional Dependency:

user_id → user_name, user_email, user_phone, user_address

Here, all the non-key attributes depend on user_id so partial dependency does not occur here. So,

- **USER – 2** (<u>user_id</u>, user_name, user_email, user_phone, user_address)

MOVIE - 1

(<u>user_id</u>, <u>movie_id</u>, movie_title, movie_duration, genre, release_date)

Composite Primary Key: (user_id, movie_id)

Non-key attributes: movie_title, movie_duration, genre, release_date

Here, Part of composite key movie_id which is part of composite primary key gives all non-key attributes so partial dependency occurs. Removing partial dependency:

- **USER - MOVIE - 2** (user_id, movie_id)
- **MOVIE - 2** (movie_id, movie_id, movie_title, movie_duration, genre, release_date)

THEATER - 1

(user_id, movie_id, theater_id, theater_name, theater_city theater_address)

Composite Primary Key: user_id, movie_id, theater_id

Non-key attributes: theater_name, theater_city theater_address

Functional Dependency:

theater_id → theater_name, theater_city theater_address

Here, Part of composite key theater_id which is part of composite primary key gives all non-key attributes so partial dependency occurs. Removing partial dependency:

- **USER - THEATER - 2** (user_id, movie_id, theater_id)
- **THEATER - 2** (theater_id, theater_name, theater_city theater_address)

HALL - 1

(user_id, movie_id, theater_id, hall_id, hall_name, capacity, hall_type)

Composite Primary Key: user_id, movie_id, theater_id, hall_id

Non-key attributes: hall_name, capacity, hall_type

Functional Dependency:

hall_id → hall_name, capacity, hall_type

Here, Part of composite key hall_id which is part of composite primary key gives all non-key attributes so partial dependency occurs. Removing partial dependency:

- **USER - HALL - 2** (user_id, movie_id, theater_id, hall_id)
- **HALL - 2** (hall_id, hall_name, capacity, hall_type)

SHOW - 1

(user_id, movie_id, theater_id, hall_id, show_id, show_name, show_time, show_date, show_price)

Composite Primary Key: user_id, movie_id, theater_id, hall_id, show_id

Non-key attributes: show_name, show_time, show_date, show_price

Functional Dependency:

show_id → show_name, show_time, show_date, show_price

Here, Part of composite key show_id which is part of composite primary key gives all non-key attributes so partial dependency occurs. Removing partial dependency:

- **USER - SHOW - 2** (user_id, movie_id, theater_id, hall_id, show_id)
- **SHOW - 2** (show_id, show_name, show_time, show_date, show_price)

TICKET - 1

(user_id, movie_id, theater_id, hall_id, show_id, ticket_id, ticket_time, seat_no, status)

Composite Primary Key: user_id, movie_id, theater_id, hall_id, show_id, ticket_id

Non-key attributes: ticket_time, seat_no, status

Functional Dependency:

ticket_id → ticket_time, seat_no, status

Here, Part of composite key ticket_id which is part of composite primary key gives all non-key attributes so partial dependency occurs. Removing partial dependency:

- **USER - TICKET - 2** (user_id, movie_id, theater_id, hall_id, show_id, ticket_id)
- **TICKET - 2** (ticket_id, ticket_time, seat_no, status)

Final 2NF tables are:

USER – 2 (user_id, user_name, user_email, user_phone, user_address)

USER - MOVIE - 2 (user_id*, movie_id*)

MOVIE - 2 (movie_id, movie_title, movie_duration, genre, release_date)

USER - THEATER - 2 (user_id*, movie_id*, theater_id*)

THEATER - 2 (theater_id, theater_name, theater_city theater_address)

USER - HALL - 2 (user_id*, movie_id*, theater_id*, hall_id*)

HALL - 2 (hall_id, hall_name, capacity, hall_type)

USER - SHOW - 2 (user_id*, movie_id*, theater_id*, hall_id*, show_id*)

SHOW - 2 (show_id, show_name, show_time, show_date, show_price)

USER - TICKET - 2 (user_id*, movie_id*, theater_id*, hall_id*, show_id*, ticket_id*)

TICKET - 2 (ticket_id, ticket_time, seat_no, status)

Anomalies and Difficulties

Second Normal Form eliminates partial dependency by making sure that non-key attributes are dependent upon the primary key. Nevertheless, despite the attainment of 2NF, some anomalies and challenges might remain:

- Dependency may also be transitive.
- Through indirect dependencies a redundancy of data can still be present.
- Transitive dependency will result in insertion anomaly.
- A possibility of update anomaly occurs when non key attributes rely on another non key attribute.
- The indirectly related data can still be lost due to deletion anomaly.

3.4. Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if:

- The relation is already in Second Normal Form (2NF), and
- No transitive dependency exists, meaning:
    - No non-key attribute depends on another non-key attribute
    - All non-key attributes depend only on the primary key

USER - 2

(user_id, user_name, user_email, user_phone, user_address)

Primary Key: user_id

Non-key attributes: user_name, user_email, user_phone, user_address

Functional Dependency:

user_id → user_name, user_email, user_phone, user_address

Transitive Dependency Check:

- No non-key attribute determines another non-key attribute
- All non-key attributes depend directly on user_id

Here, Transitive dependency does not exist, so this table is already in 3NF.

USER - MOVIE - 2

(<u>user_id</u>, <u>movie_id</u>)

Primary Key: user_id, movie_id

Non-key attributes: None

To occur transitive dependency there should be more than one non-key attribute.

So, this table is already in 3NF.

MOVIE - 2

(<u>movie_id</u>, movie_title, movie_duration, genre, release_date)

Primary Key: movie_id

Non-key attributes: movie_title, movie_duration, genre, release_date

Functional Dependency:

movie_id → movie_title, movie_duration, genre, release_date

Transitive Dependency Check:

- No dependency exists among non-key attributes
- All attributes depend solely on movie_id

Here, Transitive dependency does not exist, so this table is already in 3NF.

USER - THEATER - 2

(user_id, movie_id, theater_id)

Primary Key: user_id, movie_id, theater_id

Non-key attributes: None

To occur transitive dependency there should be more than one non-key attribute. So, this table is already in 3NF.

THEATER - 2

(theater_id, theater_name, theater_city theater_address)

Primary Key: theater_id

Non-key attributes: theater_name, theater_city, theater_address

Functional Dependency:

theater_id → theater_name, theater_city, theater_address

Transitive Dependency Check:

- No dependency exists among non-key attributes
- All non-key attributes depend only on theater_id

Here, Transitive dependency does not exist, so this table is already in 3NF.

USER - HALL - 2

(user_id, movie_id, theater_id, hall_id)

Primary Key: user_id, movie_id, theater_id, hall_id

Non-key attributes: None

To occur transitive dependency there should be more than one non-key attribute. So, this table is already in 3NF.

HALL - 2

(hall_id, hall_name, capacity, hall_type)

Primary Key: hall_id

Non-key attributes: hall_name, capacity, hall_type

Functional Dependency:

hall_id → hall_name, capacity, hall_type

Transitive Dependency Check:

- No dependency exists among non-key attributes
- All non-key attributes depend directly on hall_id

Here, Transitive dependency does not exist, so this table is already in 3NF.

USER - SHOW - 2

(user_id, movie_id, theater_id, hall_id, show_id)

Primary Key: user_id, movie_id, theater_id, hall_id, show_id

Non-key attributes: None

To occur transitive dependency there should be more than one non-key attribute. So, this table is already in 3NF.

SHOW - 2

(show_id, show_name, show_time, show_date, show_price)

Primary Key: show_id

Non-key attributes: show_name, show_time, show_date, show_price

Functional Dependency:

show_id → show_name, show_time, show_date, show_price

Transitive Dependency Check:

- No dependency exists among non-key attributes
- All non-key attributes depend directly on show_id

Here, Transitive dependency does not exist, so this table is already in 3NF.

USER - TICKET - 2

(user_id, movie_id, theater_id, hall_id, show_id, ticket_id)

Primary Key: user_id, movie_id, theater_id, hall_id, show_id, ticket_id

Non-key attributes: None

To occur transitive dependency there should be more than one non-key attribute. So, this table is already in 3NF.

TICKET - 2

(ticket_id, ticket_time, seat_no, status)

Primary Key: ticket_id

Non-key attributes: ticket_time, seat_no, status

Functional Dependency:

ticket_id → ticket_time, seat_no, status

Transitive Dependency Check:

- No non-key attribute determines another non-key attribute
- All non-key attributes depend directly on ticket_id

Here, Transitive dependency does not exist, so this table is already in 3NF.

Final 3NF Tables are:

USER – 3 (user_id, user_name, user_email, user_phone, user_address)

USER - MOVIE - 3 (user_id*, movie_id*)

MOVIE - 3 (movie_id, movie_title, movie_duration, genre, release_date)

USER - THEATER - 3 (user_id*, movie_id*, theater_id*)

THEATER - 3 (theater_id, theater_name, theater_city theater_address)

USER - HALL - 3 (user_id*, movie_id*, theater_id*, hall_id*)

HALL - 3 (hall_id, hall_name, capacity, hall_type)

USER - SHOW - 3 (user_id*, movie_id*, theater_id*, hall_id*, show_id*)

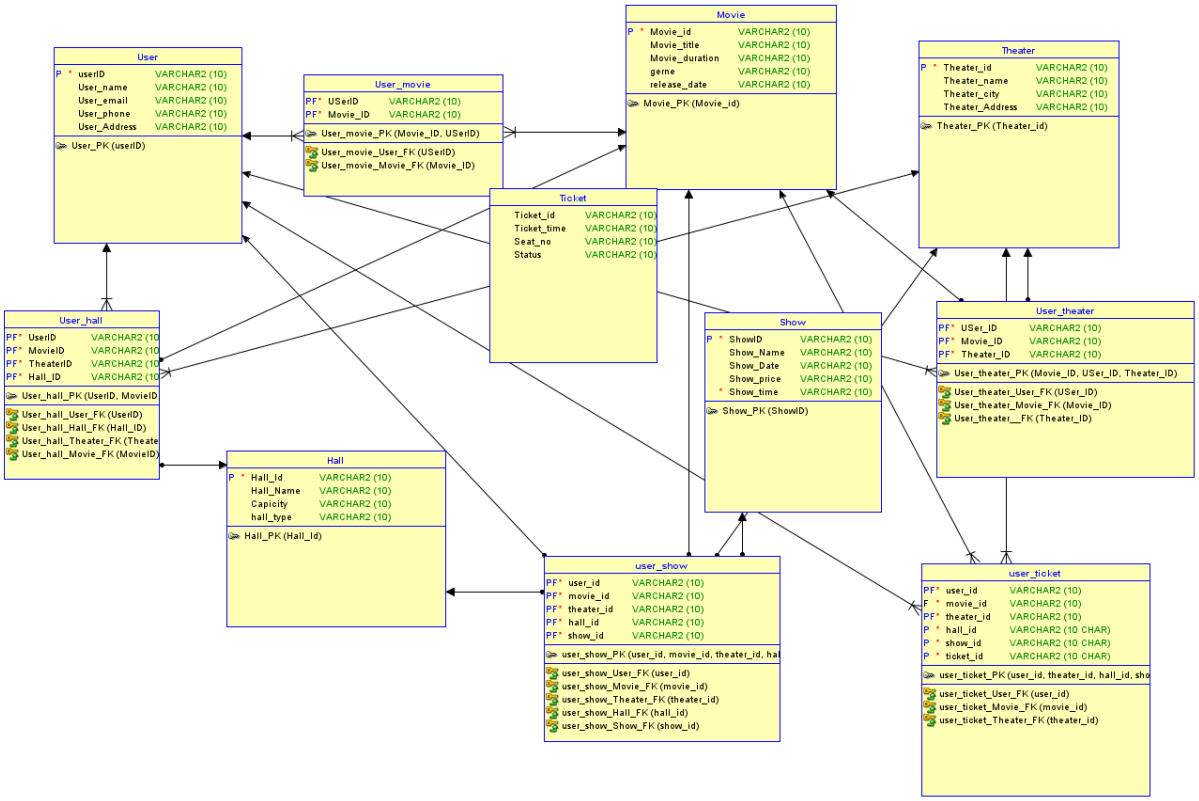
SHOW - 3 (show_id, show_name, show_time, show_date, show_price)

USER - TICKET - 3 (user_id*, movie_id*, theater_id*, hall_id*, show_id*, ticket_id*)

TICKET - 3 (ticket_id, ticket_time, seat_no, status)

Anomalies and Difficulties

Third Normal Form eliminates transitive dependency and makes all the non-key attributes dependent on the primary key. Once the 3NF has been reached, most data anomalies are eliminated, still, some practical challenges might persist:

- Increased number of tables
- More joins are necessary in queries.
- Minor complexity of query.

4. Final ERD